Automation with AI and MCP Servers using Playwright-Notes-1

- **AI Assistant-** An AI assistant is a computer program powered by artificial intelligence that can understand your questions and respond in a helpful, human-like way- an intelligent helper tool or feature that helps users by understanding natural language and providing meaningful responses or actions. Eg: Claude, ChatGPT, Gemini, Copilot

- **AI Model-** a program that has been trained on massive amounts of data to recognize patterns and make predictions or generate responses- brain behind the assistant-understands natural language and generates responses

  Simple analogy:

  Think of a model like a student who has read millions of books, articles, and websites. After all that reading, the student has developed the ability to answer questions, write, and explain things — not by looking things up, but by remembering patterns from everything they studied.

  That "trained student" is the model.

  How a model is created simply:
  Step 1 → Feed massive amounts of text data to the program
  (books, websites, articles, code, conversations)
  ↓
  Step 2 → Program learns patterns, relationships between words,
  grammar, facts, logic, reasoning
  ↓
  Step 3 → Program is tested and fine tuned
  ↓
  Step 4 → Final trained program = MODEL
  ↓
  Step 5 → You interact with it

Real world analogy:

Thing	AI equivalent
Student studying for years	**Training the model**
All the books student read	**Training data**
Student's brain after studying	**The model**
Student answering your question	**Model generating a response**

Types of AI models:

Model type	What it does	Example
Language model	Understands and generates text	Claude, ChatGPT
Image model	Generates or recognizes images	DALL-E, Midjourney
Speech model	Understands or generates audio	Whisper
Code model	Writes and understands code	GitHub Copilot

Model vs AI Assistant:

	Model	AI Assistant
What it is	The **brain** — raw trained program	The **interface** — product built on top of model
Can you talk to it directly	⚠️ Usually through an interface	✅ Yes
Example	Claude Sonnet 4.6	Claude.ai chat

The **model** is the engine. The **AI assistant** is the car built around that engine.

Bottom line:

A model is a **program that learned from huge amounts of data** and can now understand language, answer questions, write, and reason — all based on patterns it absorbed during training.

- **AI Agent-** a program that can think, plan, and take actions on its own to complete a goal — not just answer a single question-a worker that uses the AI model (brain) to do the job

How an agent works:

You give a GOAL
↓
Agent THINKS — what steps do I need?
↓
Agent PLANS — step 1, step 2, step 3...
↓
Agent ACTS — searches web, runs code, reads files
↓
Agent OBSERVES — did that work? what's next?
↓
Agent REPEATS until goal is complete
↓
Agent gives you FINAL RESULT

What tools can an agent use:

```
python# An agent can use many tools automatically
tools = [
    "web_search",      # search the internet
    "run_code",        # execute python code
    "read_file",       # read documents
    "send_email",      # send emails
    "browse_website",  # open and read websites
    "call_api",        # talk to external services
    "write_file",      # save files
]
```

Model vs Assistant vs Agent:

	Model	Assistant	Agent
What it is	The brain	Answers questions	Takes actions
Interaction	One response	Back and forth chat	Autonomous steps
Uses tools	❌ No	⚠️ Sometimes	✅ Yes
Works independently	❌ No	❌ No	✅ Yes
Example	Claude Sonnet 4.6	Claude chat	Claude Code

Bottom line:

A regular AI **answers**. An AI agent **acts**.

You give an agent a **goal**, not just a question — and it figures out all the steps, uses tools, and completes the task on its own.

- **MCP(Model Context Protocol)-** open or standard protocol that enables developers/engineers to build secure 2 way connections between their data sources and AI Powered tools- **It is a standard way for AI models to connect and talk to external tools and services-**acts as a bridge between the AI model and the data sources, websites,apps, tools,files

Simple analogy:

Think of MCP like a **universal remote control**.

Before universal remotes existed, every device — TV, DVD player, AC — had its own separate remote. It was messy and inconsistent.

MCP is like the universal remote — **one standard protocol that lets AI connect to ANY tool or service in the same consistent way**.

The problem MCP solves:

Before MCP:

AI wanted to use Google Drive → needed custom code
AI wanted to use Gmail → needed different custom code
AI wanted to use Slack → needed completely different code
AI wanted to use GitHub → needed yet another custom code

Every tool = different connection = messy and inconsistent

After MCP:

AI wanted to use Google Drive → uses MCP ✅
AI wanted to use Gmail → uses MCP ✅
AI wanted to use Slack → uses MCP ✅
AI wanted to use GitHub → uses MCP ✅

Every tool = same standard connection = clean and consistent

The AI doesn't need to learn a new language for each tool — MCP is the **one common language** every tool speaks.

What MCP enables AI to do:

With MCP connected tools, AI can:

📧 Gmail → read emails, send replies, search inbox
📅 Calendar → check schedule, create events
📁 Google Drive → find files, read documents
💬 Slack → read messages, send updates
🐙 GitHub → read code, create issues
📊 Spreadsheets → read and update data

MCP in simple steps:

Step 1 → Tool developer builds an MCP server for their tool
↓
Step 2 → You connect that MCP server to your AI
↓
Step 3 → AI can now use that tool automatically
↓
Step 4 → You just talk to AI naturally
"Find my latest email from John"
↓
Step 5 → AI uses MCP to talk to Gmail and gets the answer

MCP vs no MCP:

WITHOUT MCP:
You: "Find my latest email from John"
AI: "I cannot access your email"

WITH MCP (Gmail connected):
You: "Find my latest email from John"
AI: searches Gmail via MCP
AI: "John emailed you yesterday about the project deadline"

Bottom line:

Concept	Simple meaning
MCP	Universal standard for AI to connect to tools
MCP Server	A tool's plug that speaks the MCP language
MCP Client	The AI that uses the MCP connection

MCP is simply a **universal plug** that lets AI assistants connect to and use any external tool or service — emails, files, calendars, code, and more — all through one consistent standard.

- **MCP Client-** AI assistant or frontend tool that sends structured commands
- **MCP Server-** Backend system that receives commands and performs real-world actions-the application/service that implements the MCP

Simple analogy:

Think of a **restaurant**:

- The **waiter** takes your order and carries it to the kitchen — that's the **MCP Client**
- The **kitchen** receives the order, prepares the food, and sends it back — that's the **MCP Server**

You (the user) just talk to the waiter. The waiter handles all communication with the kitchen.

MCP Client

The MCP client is the **AI side** — it **sends requests** to MCP servers asking for things.

You say: "Find my latest email from John"

↓

MCP Client (Claude) thinks:

"I need to search Gmail — let me send a request to the Gmail MCP server"

↓

MCP Client sends request → Gmail MCP Server

The client is the **asker** — it asks tools to do things on behalf of you.

MCP Server

The MCP server is the **tool side** — it **receives requests**, does the work, and **sends back results**.

Gmail MCP Server receives request:

"Search for latest email from John"

↓

Server searches Gmail

↓

Server sends back result → MCP Client (Claude)

↓

Claude tells you: "John emailed you yesterday about the deadline"

The server is the **doer** — it does the actual work and returns results.

How they work together:

YOU

↓ "Find my latest email from John"

MCP CLIENT (Claude)

↓ sends request in MCP standard language

MCP SERVER (Gmail)

↓ searches Gmail, finds email

MCP CLIENT (Claude)

↓ receives result, understands it

YOU

↑ "John emailed you yesterday about the deadline"

Real world example with Google Drive:

You: "Find my Q3 sales presentation"

↓

MCP Client: sends request to Google Drive MCP Server

"search files for Q3 sales presentation"

↓

MCP Server: searches Google Drive

finds the file

sends back file details

↓

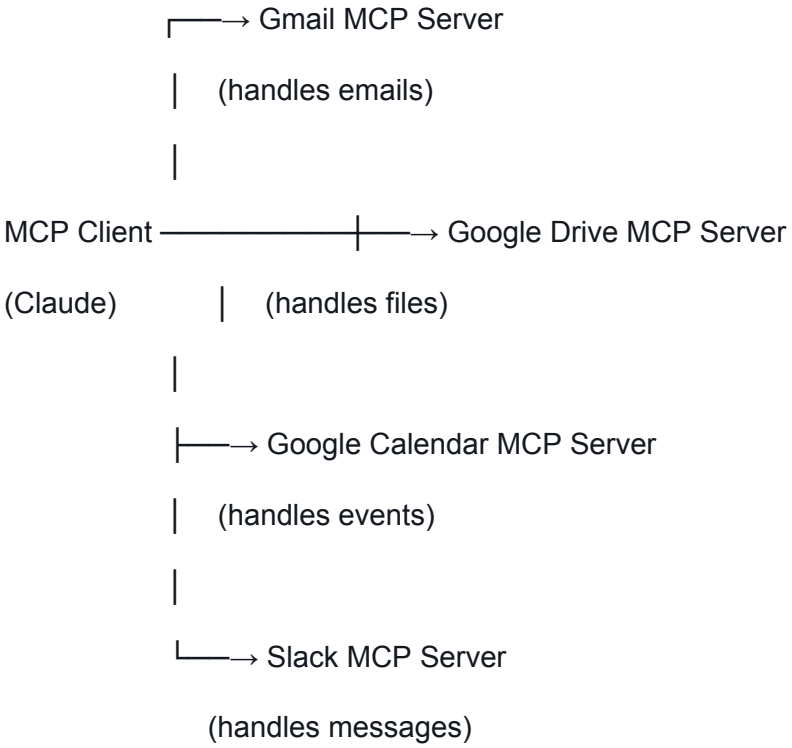
MCP Client: receives the result

↓

You: "Found it — Q3 Sales Results.pptx, last edited Monday"

Multiple MCP Servers at once:

The same MCP Client can talk to **many servers simultaneously**.



You just talk to Claude normally — Claude figures out which server to talk to.

Example using multiple servers:

You: "Check if I have any emails about the meeting

and add it to my calendar"

MCP Client (Claude):

Step 1 → talks to Gmail MCP Server → finds meeting email

Step 2 → reads email details → Monday 3pm with team

Step 3 → talks to Calendar MCP Server → creates calendar event

Step 4 → tells you everything is done

Summary table:

	MCP Client	MCP Server
What it is	The AI	The tool
Role	Sends requests	Receives and handles requests
Thinks and decides	✅ Yes	❌ No — just executes
Does the actual work	❌ No	✅ Yes

The client **asks**. The server **does**. You just **talk naturally** — MCP handles everything in between.

Examples of MCP Servers and MCP Clients

MCP Clients

MCP Clients are the AI side — the applications that connect to MCP servers and use them.

Claude Desktop is the most widely used MCP client. It is Anthropic's desktop application that can connect to any MCP server you configure, giving Claude the ability to access your files, emails, calendar, and more directly from the chat interface.

Claude.ai is the web version of Claude that supports MCP connections, allowing you to connect external tools and services directly in your browser based chat.

Cursor is an AI powered code editor that acts as an MCP client, connecting to MCP servers to give the AI inside it access to databases, APIs, documentation, and external tools while you code.

Continue is a VS Code extension that acts as an MCP client, letting the AI assistant inside your code editor connect to external tools and services.

Claude Code is Anthropic's command line tool that acts as an MCP client, connecting to servers to give Claude the ability to interact with your codebase, run commands, and use external tools.

Zed is a code editor that supports MCP, acting as a client that connects AI capabilities with external tools and services.

MCP Servers

MCP Servers are the tool side — the services that receive requests from clients and do the actual work.

File and Storage Servers

File System MCP Server lets Claude read, write, search, and manage files and folders on your local computer. You can ask Claude to read a document, create a new file, or search for files containing specific words.

Google Drive MCP Server connects Claude to your Google Drive account, allowing it to search for documents, read file contents, list folders, and retrieve specific files you ask for.

AWS S3 MCP Server gives Claude access to Amazon S3 cloud storage buckets, letting it upload, download, list, and manage files stored in the cloud.

Communication Servers

Gmail MCP Server connects Claude to your Gmail account so it can read emails, search your inbox, find specific messages, and send replies on your behalf.

Slack MCP Server lets Claude read and send messages in your Slack workspace, search conversations, and interact with your team channels.

Microsoft Outlook MCP Server gives Claude access to your Outlook emails and lets it search, read, and manage your inbox.

Productivity Servers

Google Calendar MCP Server connects Claude to your Google Calendar so it can check your schedule, find upcoming events, create new events, and manage your appointments.

Notion MCP Server lets Claude read and write pages in your Notion workspace, search your notes, and update databases.

Todoist MCP Server gives Claude access to your task manager so it can create tasks, check what is pending, mark tasks complete, and manage your to do lists.

Development Servers

GitHub MCP Server connects Claude to GitHub, letting it read code repositories, search files, create issues, read pull requests, and interact with your codebase.

GitLab MCP Server does the same for GitLab repositories — reading code, managing issues, and interacting with merge requests.

PostgreSQL MCP Server lets Claude connect to your database, run queries, read table structures, and retrieve data directly.

SQLite MCP Server gives Claude access to local SQLite databases so it can query, read, and interact with your local data.

Browser and Web Servers

Playwright MCP Server gives Claude the ability to control a real web browser — navigating websites, clicking buttons, filling forms, taking screenshots, and extracting content from pages.

Puppeteer MCP Server is similar to Playwright and lets Claude automate Chrome browser actions like navigation, clicking, and scraping.

Fetch MCP Server lets Claude make HTTP requests to any URL, fetch webpage content, and call web APIs directly.

Business Tool Servers

Asana MCP Server connects Claude to your Asana workspace so it can read tasks, create new ones, check project status, and update assignments.

Jira MCP Server lets Claude interact with your Jira board — reading tickets, creating issues, updating statuses, and tracking bugs.

Salesforce MCP Server gives Claude access to your Salesforce CRM so it can read customer data, update records, and search contacts.

HubSpot MCP Server connects Claude to HubSpot so it can manage contacts, deals, and marketing data.

AI and Search Servers

Brave Search MCP Server lets Claude perform real web searches using Brave's search engine and return current results.

Google Maps MCP Server gives Claude the ability to search for places, get directions, find nearby locations, and retrieve location information.

Memory MCP Server gives Claude a persistent memory store so it can remember information across conversations that it can save and retrieve later.

How they connect in real life:

One MCP Client like Claude Desktop can connect to many servers at the same time. For example you could have Claude connected to Gmail, Google Drive, Google Calendar, GitHub, and Slack all at once. Then in a single conversation you could say:

"Check my emails for anything about the project, find the related document in Drive, check if I have a meeting about it on Calendar, and post a summary to the team Slack channel"

Claude would automatically talk to all four MCP servers — Gmail, Drive, Calendar, and Slack — to complete that one request without you having to switch between apps at all.

Bottom line

MCP Clients are the AI applications that want to use tools. MCP Servers are the tools themselves. Any client can connect to any server as long as both speak the MCP standard — making the whole system incredibly flexible and expandable.

<u>Eg: Filesystem MCP Server working</u>

You: Delete .log files older than 30 days

⬇️

AI Assistant: Understands the query in natural language and passes the request to the model

⬇️

Model: understands the intent/request and plans safe file deletion logic-triggers a task for the agent

⬇️

Agent: prepares a Python script for the task- checks access rules-accesses system securely through MCP-like protocol (file system bridge)-enforces access control (read-only deletable) and provides a safe sandbox to operate in

⬇️

Files older than 30 days deleted safely

Eg: Playwright MCP Server- a MCP server that provides browser automation capabilities using Playwright. The server enables LLMs to interact with webpages through structured accessibility screenshots, bypassing the need for screenshots or visually-tuned models-Think of Playwright MCP Server like a remote control for a web browser.

Normally you sit in front of a browser and click, type, scroll yourself.

Playwright MCP Server lets Claude sit in front of the browser and do all of that automatically on your behalf.

It is an **MCP server that gives Claude the ability to control a real web browser** — clicking, typing, navigating, taking screenshots, and scraping data — all through natural conversation.

You tell Claude: "Go to saucedemo.com and login"

↓

Claude uses: Playwright MCP Server

↓

Server controls: Real Chrome browser

↓

Browser does: Opens site, types credentials, clicks login

↓

Claude tells you: "Successfully logged in"

What Playwright MCP Server can do:

🌐 Navigate → go to any URL

🖱️ Click → click buttons, links, checkboxes

⌨️ Type → fill forms, search boxes

📸 Screenshot → capture what the browser sees

📜 Scroll → scroll up, down, anywhere

🔍 Find elements → locate elements on the page

📋 Read content → extract text from the page

⏳ Wait → wait for elements to load

🔙 Go back → navigate browser history

How it works step by step:

YOU

↓ "Login to saucedemo with standard_user"

MCP CLIENT (Claude)

↓ sends instructions to Playwright MCP Server

PLAYWRIGHT MCP SERVER

↓ controls real Chrome browser

↓ Step 1 → opens https://www.saucedemo.com

↓ Step 2 → finds username field

↓ Step 3 → types standard_user

↓ Step 4 → finds password field

↓ Step 5 → types secret_sauce

↓ Step 6 → clicks login button

↓ Step 7 → takes screenshot

↓ sends back result + screenshot

MCP CLIENT (Claude)

↓ receives result

YOU

↑ "Successfully logged in. Here's a screenshot of the dashboard"

Eg: Ordering a laptop from Amazon

You: I want to order a laptop from Amazon

⬇️

AI Assistant: Listens to and understands your request-passes request to model

⬇️

Model: Interprets the request/intent-plans steps:search->filter->order-breaks down tasks for agent

⬇️

Agent: Uses browser automation (Playwright)->Prepares actions:search->click->fill form- interacts with Amazon website via MCP

⬇️

MCP (Playwright MCP Server for Amazon): Provides structured web layout (DOM elements)- allows secure automation with cookies/session storage

Flow: MCP helps Agent: Search laptop under rupees 60,000->Click the right product->add to cart->sign in securely(cookie/session preloaded)->simulate checkout/reach payment page

⬇️

Result: You're about to order Lenovo Ideapad Slim 3 -rupees 58,700 Item added to cart, ready for checkout

Refer to https://modelcontextprotocol.io/docs/sdk for more information